

**PW INSTITUTE OF INNOVATION**

*Department of Computer Science*

A DBMS Project Report on

**BUS RESERVATION SYSTEM**

*"JourneyJet" — Manage Passengers, Buses, Conductors and Tickets*

**Submitted by**

Samarth Swami

Nikitha Kodithyala

Safia Hana

Bhavya Kumar

Submitted in partial fulfillment of the requirements for the degree of

**BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE**

**Guide**

Mr. Rishav Upadhyaya

**Academic Session**

2025 – 2029

## Certificate

---

This is to certify that the project entitled "Bus Reservation System (JourneyJet)" has been submitted by Samarth Swami, Nikitha Kodithyala, Safia Hana, and Bhavya Kumar, students of B.Tech Computer Science, PW Institute of Innovation, as a bonafide record of work carried out under my guidance in partial fulfillment of the requirements for the Database Management System course during the academic session 2025-2029.

The project has been examined and found satisfactory in terms of database design, normalization, SQL implementation, and overall system functionality.

Date: \_\_\_\_\_

Place: PW Institute of Innovation

---

**Mr. Rishav Upadhyaya**  
*Guide, Department of Computer Science*

## Acknowledgement

---

We take this opportunity to express our sincere gratitude to our guide, Mr. Rishav Upadhyaya, for his constant guidance, encouragement, and valuable suggestions throughout the duration of this Database Management System project. His insights into relational database design and SQL implementation were instrumental in shaping this work.

We are also thankful to the Department of Computer Science, PW Institute of Innovation, for providing us with the necessary resources, infrastructure, and a conducive environment to carry out this project successfully.

We would like to extend our appreciation to our teammates for their collaborative effort in database design, query writing, backend implementation, and documentation, which made this project a comprehensive learning experience.

Finally, we thank our family and friends for their continuous support and motivation throughout this project.

## Abstract

---

The Bus Reservation System, developed under the name “JourneyJet,” is a database-driven application designed to automate the management of passengers, buses, conductors, and ticket bookings for a transportation service. The project replaces error-prone manual record-keeping with a centralized, relational database that ensures data accuracy, consistency, and quick retrieval.

The system is built using MySQL as the backend database, with a Node.js/Express layer exposing the data through a structured API, and a browser-based frontend that allows staff to add, update, delete, and search records across all major modules: Passenger, Bus, Conductor, and Ticket. A dedicated Reports module demonstrates the use of SQL joins to combine data across multiple related tables — for example, linking passengers to the buses they travel on, and conductors to the buses and tickets they manage.

Database integrity is maintained through the use of primary keys, foreign keys, and constraints, while the schema itself has been normalized up to the Third Normal Form (3NF) to eliminate redundancy and update anomalies. The project currently manages 100+ passengers, 100 buses, 100 conductors, and 100 tickets, and supports CRUD operations, aggregate reporting, and multi-table queries across all entities.

Overall, JourneyJet demonstrates the practical application of core DBMS concepts — ER modeling, relational schema design, normalization, and SQL querying — within a working, end-to-end software system.

**Keywords:** Database Management System, MySQL, Bus Reservation, SQL, ER Model, Normalization, CRUD Operations.

# Table of Contents

---

|                                             |           |
|---------------------------------------------|-----------|
| <b>Certificate.....</b>                     | <b>2</b>  |
| <b>Acknowledgement.....</b>                 | <b>3</b>  |
| <b>Abstract.....</b>                        | <b>4</b>  |
| <b>Chapter 1: Introduction.....</b>         | <b>6</b>  |
| 1.1 Overview.....                           | 6         |
| 1.2 Need for the System.....                | 6         |
| 1.3 Objectives.....                         | 6         |
| <b>Chapter 2: Requirement Analysis.....</b> | <b>7</b>  |
| 2.1 Functional Requirements.....            | 7         |
| 2.2 Non-Functional Requirements.....        | 8         |
| <b>Chapter 3: System Study.....</b>         | <b>9</b>  |
| 3.1 Existing System.....                    | 9         |
| 3.2 Proposed System.....                    | 9         |
| <b>Chapter 4: System Design.....</b>        | <b>10</b> |
| 4.1 Architecture.....                       | 10        |
| 4.2 Modules.....                            | 10        |
| <b>Chapter 5: Database Design.....</b>      | <b>13</b> |
| 5.1 Entity-Relationship Diagram.....        | 13        |
| 5.2 Entities and Key Attributes.....        | 13        |
| 5.3 Relationships.....                      | 14        |
| <b>Chapter 6: Relational Schema.....</b>    | <b>15</b> |
| <b>Chapter 7: Normalization.....</b>        | <b>17</b> |
| <b>Chapter 8: SQL Implementation.....</b>   | <b>18</b> |
| 8.1 CREATE TABLE Statements.....            | 18        |
| 8.2 Sample Queries.....                     | 20        |
| <b>Chapter 9: Testing.....</b>              | <b>21</b> |
| <b>Chapter 10: Results.....</b>             | <b>21</b> |
| <b>Future Scope.....</b>                    | <b>23</b> |
| <b>Conclusion.....</b>                      | <b>23</b> |
| <b>References.....</b>                      | <b>24</b> |

## Chapter 1: Introduction

---

### 1.1 Overview

Transportation management plays a critical role in modern society. Traditional bus reservation methods often rely on manual record-keeping, which leads to inefficiencies, duplicate entries, and inconsistent data across registers. As the volume of passengers and routes grows, these manual systems become increasingly difficult to scale and maintain.

The Bus Reservation System, JourneyJet, aims to digitize reservation management through a structured relational database. It centralizes information about passengers, buses, conductors, and tickets into a single, normalized MySQL database, accessible through a simple web-based dashboard.

### 1.2 Need for the System

- Reduce manual paperwork and human error in record-keeping
- Improve data accuracy and consistency across passenger, bus, and ticket records
- Enable faster ticket processing and seat/fare lookups
- Provide better passenger and conductor management
- Generate instant reports using SQL joins and aggregate functions
- Maintain referential integrity across all related entities

### 1.3 Objectives

1. Design a normalized relational database for the bus reservation domain
2. Manage passenger records efficiently (add, update, delete, search)
3. Store and maintain bus information including routes, timing, and seating
4. Maintain conductor records and their assignment to buses
5. Issue, update, and track tickets with fare and seat details
6. Generate consolidated reports using SQL joins across multiple tables
7. Demonstrate core DBMS concepts — ER modeling, normalization, constraints, and querying — in a practical, working system

## Chapter 2: Requirement Analysis

---

### 2.1 Functional Requirements

#### *Passenger Module*

- Add Passenger
- Update Passenger
- Delete Passenger
- Search Passenger

### **Bus Module**

- Add Bus
- Update Bus
- Delete Bus
- Search Bus

### **Conductor Module**

- Add Conductor
- Update Conductor
- Delete Conductor

### **Ticket Module**

- Create Ticket
- Update Ticket
- Delete Ticket

### **Report Module**

- Passenger Ticket Details (Passenger ✕ Ticket join)
- Passenger Bus Details (Passenger ✕ Bus join)
- Conductor Bus Details (Conductor ✕ Bus join)

## **2.2 Non-Functional Requirements**

### **2.2.1 Performance**

The system should provide fast query execution, with dashboard statistics and table listings loading within a few seconds even as the dataset scales to hundreds of records.

### **2.2.2 Usability**

The interface is designed to be simple and self-explanatory, allowing any staff member to add, search, and manage records without prior technical training.

### **2.2.3 Reliability**

Data should remain consistent across all modules at all times, with foreign key constraints preventing orphaned or invalid references between passengers, buses, conductors, and tickets.

### **2.2.4 Security**

Database-level constraints (NOT NULL, UNIQUE, foreign keys) must prevent invalid or duplicate entries from being inserted into the system.

### **2.2.5 Scalability**

The relational schema is designed so that new modules (e.g., online payments, seat maps) can be added in the future without disrupting the existing structure.

## Chapter 3: System Study

---

### 3.1 Existing System

In a traditional, manual bus reservation process, records are maintained on paper registers or disconnected spreadsheets. This approach suffers from several limitations:

- Paper-based records that are difficult to search and cross-reference
- High likelihood of human error during data entry
- Slow searching when looking up a specific passenger, bus, or ticket
- Difficulty generating consolidated reports across passengers, buses, and conductors
- No enforced data integrity — duplicate or inconsistent records are common

### 3.2 Proposed System

JourneyJet replaces this manual process with a centralized, relational database system that provides:

- Automated record management through a structured web dashboard
- A centralized MySQL database with enforced primary and foreign key constraints
- SQL-based reporting using joins, aggregate functions, and views
- Full CRUD (Create, Read, Update, Delete) operations for every entity
- A consistent, normalized schema (up to 3NF) that eliminates redundancy

## Chapter 4: System Design

### 4.1 Architecture

JourneyJet follows a simple three-tier architecture:

```
Frontend (HTML / CSS / JavaScript)
    ↓
Backend API (Node.js / Express)
    ↓
MySQL Database
```

The frontend communicates with the backend over HTTP using `fetch()` calls to REST-style endpoints. The Express server validates requests and executes parameterized SQL queries against the MySQL database, returning JSON responses that are rendered into the dashboard tables shown below.

### 4.2 Modules

#### 4.2.1 Dashboard

The Dashboard module provides a quick, at-a-glance summary of the system's current state, displaying live counts for total passengers, buses, conductors, and tickets.

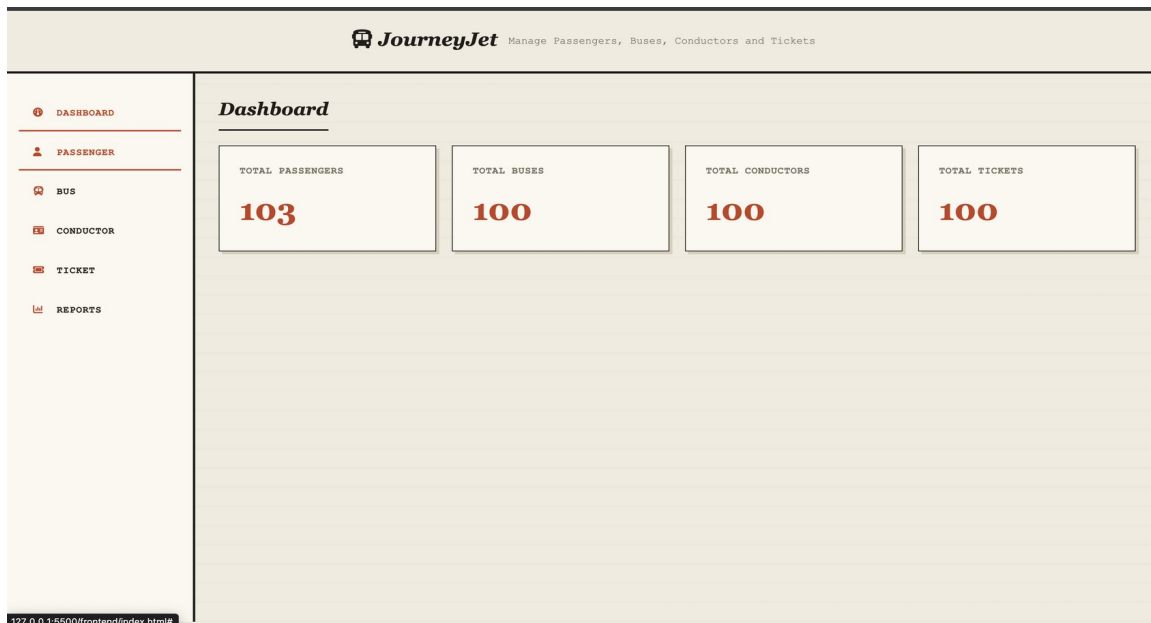


Fig 4.1: Dashboard — summary view of Total Passengers, Buses, Conductors and Tickets

#### 4.2.2 Passenger Management

This module allows staff to add new passengers, update existing details, delete records, and search through the passenger list using fields such as name, gender, age, phone, email, and address.

**Passenger Management**

First Name Last Name Select Gender Age

Phone Number Email Address Address

ADD UPDATE DELETE CLEAR

Search Passenger...

| ID | FIRST NAME | LAST NAME | GENDER | AGE | PHONE      | EMAIL            | ADDRESS     |
|----|------------|-----------|--------|-----|------------|------------------|-------------|
| 1  | Rahul      | Sharma    | Male   | 19  | 9000000001 | rahul1@gmail.com | Bhubaneswar |
| 2  | Priya      | Sharma    | Female | 20  | 9000000002 | priya2@gmail.com | Cuttack     |
| 3  | Amit       | Sharma    | Male   | 21  | 9000000003 | amit3@gmail.com  | Berhampur   |
| 4  | Sneha      | Sharma    | Female | 22  | 9000000004 | sneha4@gmail.com | Rourkela    |
| 5  | Rohit      | Sharma    | Male   | 23  | 9000000005 | rohit5@gmail.com | Puri        |
| 6  | Pooja      | Sharma    | Female | 24  | 9000000006 | pooja6@gmail.com | Sambalpur   |

Fig 4.2: Passenger Management — form, search bar, and passenger records table

### 4.2.3 Bus Management

The Bus module manages bus details including bus number, name, type (AC / Non-AC / Sleeper / Semi-Sleeper), total seats, source, destination, and departure/arrival timings.

**Bus Management**

Bus Number Bus Name Select Bus Type Total Seats

Source Destination --:-- -- --:-- --

ADD UPDATE DELETE CLEAR

Search Bus...

| BUS NO.    | BUS NAME  | TYPE         | SEATS | SOURCE      | DESTINATION | DEPARTURE | ARRIVAL  |
|------------|-----------|--------------|-------|-------------|-------------|-----------|----------|
| OD01BUS001 | Express 1 | AC           | 50    | Bhubaneswar | Puri        | 07:00:00  | 09:30:00 |
| OD02BUS002 | Express 2 | Non-AC       | 50    | Cuttack     | Rourkela    | 08:00:00  | 10:30:00 |
| OD03BUS003 | Express 3 | Sleeper      | 50    | Berhampur   | Sambalpur   | 09:00:00  | 11:30:00 |
| OD04BUS004 | Express 4 | Semi-Sleeper | 50    | Puri        | Balasore    | 10:00:00  | 12:30:00 |
| OD05BUS005 | Express 5 | AC           | 50    | Rourkela    | Angul       | 11:00:00  | 13:30:00 |
| OD06BUS006 | Express 6 | Non-AC       | 50    | Sambalpur   | Jeypore     | 12:00:00  | 14:30:00 |

Fig 4.3: Bus Management — route, timing, and seat configuration

#### 4.2.4 Conductor Management

This module maintains conductor records, including name, contact details, and years of experience, with full Add / Update / Delete / Search functionality.

| ID | FIRST NAME | LAST NAME | PHONE      | EMAIL           | EXPERIENCE |
|----|------------|-----------|------------|-----------------|------------|
| 1  | Raj        | Patel     | 8000000001 | cond1@gmail.com | 1          |
| 2  | Sunil      | Patel     | 8000000002 | cond2@gmail.com | 2          |
| 3  | Kiran      | Patel     | 8000000003 | cond3@gmail.com | 3          |
| 4  | Ajay       | Patel     | 8000000004 | cond4@gmail.com | 4          |
| 5  | Ramesh     | Patel     | 8000000005 | cond5@gmail.com | 5          |
| 6  | Vijay      | Patel     | 8000000006 | cond6@gmail.com | 6          |

Fig 4.4: Conductor Management — conductor records and contact details

#### 4.2.5 Ticket Management

The Ticket module issues and tracks ticket bookings, capturing ticket number, booking date, journey date, seat number, and fare for each transaction.

| ID | TICKET NO. | BOOKING DATE             | JOURNEY DATE             | SEAT NO. | FARE   |
|----|------------|--------------------------|--------------------------|----------|--------|
| 1  | TKT00001   | 2026-06-02T00:00:00.000Z | 2026-07-02T00:00:00.000Z | S1       | 375.00 |
| 2  | TKT00002   | 2026-06-03T00:00:00.000Z | 2026-07-03T00:00:00.000Z | S2       | 400.00 |
| 3  | TKT00003   | 2026-06-04T00:00:00.000Z | 2026-07-04T00:00:00.000Z | S3       | 425.00 |
| 4  | TKT00004   | 2026-06-05T00:00:00.000Z | 2026-07-05T00:00:00.000Z | S4       | 450.00 |
| 5  | TKT00005   | 2026-06-06T00:00:00.000Z | 2026-07-06T00:00:00.000Z | S5       | 475.00 |
| 6  | TKT00006   | 2026-06-07T00:00:00.000Z | 2026-07-07T00:00:00.000Z | S6       | 500.00 |

Fig 4.5: Ticket Management — booking and journey details with fare tracking

#### 4.2.6 Reports Module

The Reports module demonstrates the relational power of the database by joining multiple tables together — Passenger Ticket Details, Passenger Bus Details, and Conductor Bus Details — to produce consolidated, cross-entity views.

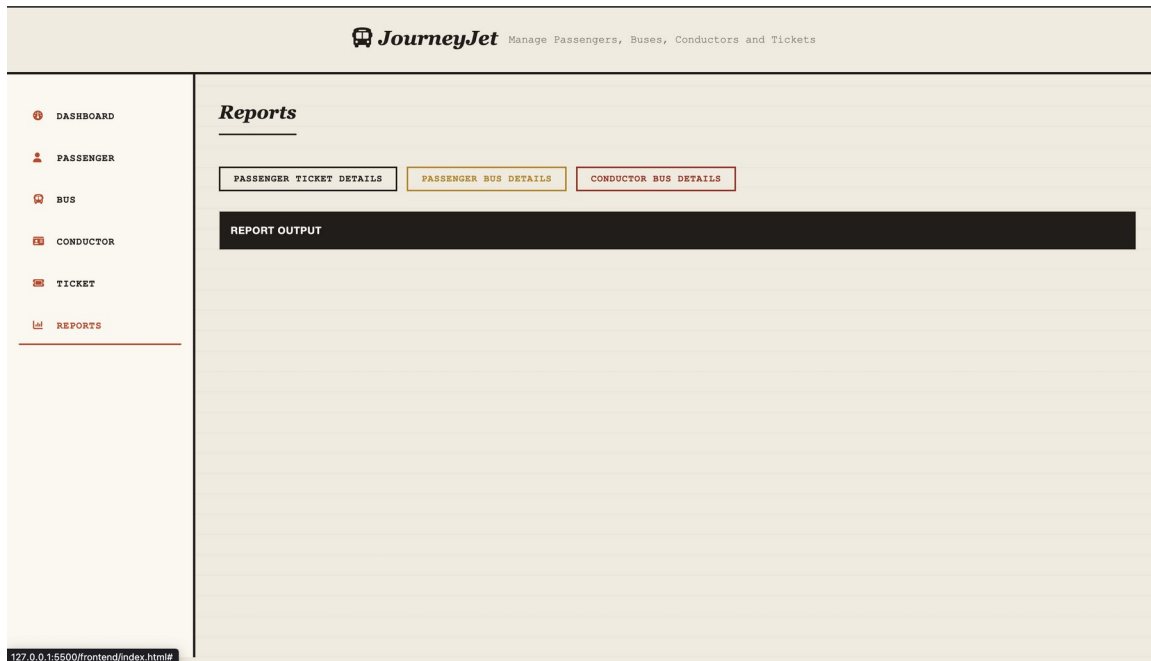


Fig 4.6: Reports — cross-table report generation via SQL joins

## Chapter 5: Database Design

### 5.1 Entity-Relationship Diagram

The ER diagram below models four core entities — Passenger, Bus, Conductor, and Ticket — connected through four relationships: Buy (Passenger–Ticket), Travel (Passenger–Bus), Have (Conductor–Bus), and Check (Conductor–Ticket).

#### E-R Diagram of Bus Reservation System:

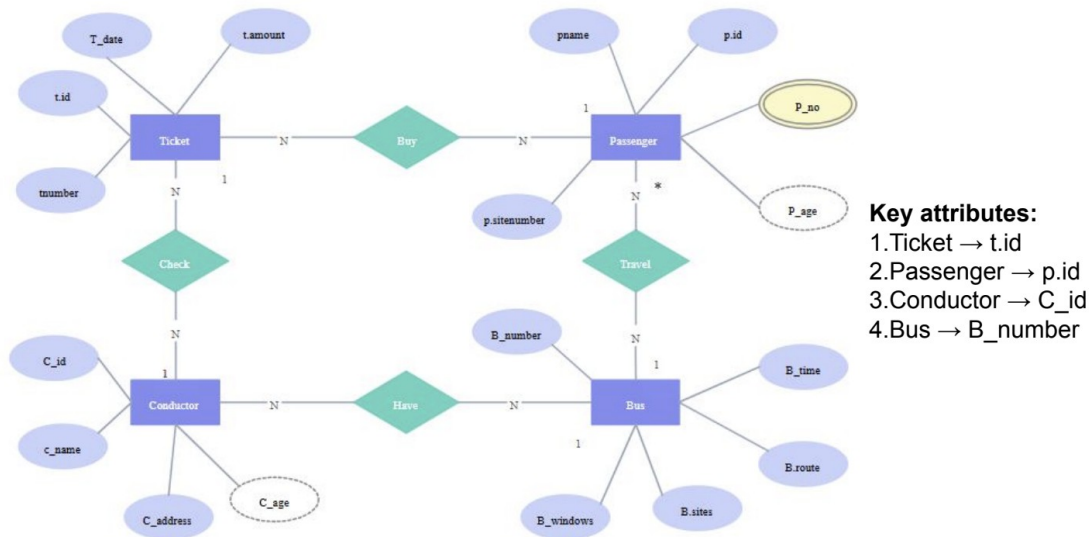


Fig 5.1: E-R Diagram of the Bus Reservation System (JourneyJet)

### 5.2 Entities and Key Attributes

#### Passenger

- p\_id (Primary Key)
- pname
- p\_age
- p\_no
- p\_sitenumber

#### Bus

- B\_number (Primary Key)
- B\_route
- B\_time
- B\_sites
- B\_windows

**Conductor**

- C\_id (Primary Key)
- C\_name
- C\_address
- C\_age

**Ticket**

- t\_id (Primary Key)
- tnumber
- T\_date
- tamount

**5.3 Relationships**

| Relationship | Participating Entities | Cardinality |
|--------------|------------------------|-------------|
| Buy          | Passenger ↔ Ticket     | N : N       |
| Travel       | Passenger ↔ Bus        | N : 1       |
| Have         | Conductor ↔ Bus        | N : N       |
| Check        | Conductor ↔ Ticket     | N : 1       |

## Chapter 6: Relational Schema

The conceptual ER model was translated into the following relational schema, implemented directly in MySQL. Each table below corresponds to one entity from the ER diagram.

### 6.1 passengers

| Column     | Type                          | Constraint                  |
|------------|-------------------------------|-----------------------------|
| p_id       | INT                           | PRIMARY KEY, AUTO_INCREMENT |
| first_name | VARCHAR(50)                   | NOT NULL                    |
| last_name  | VARCHAR(50)                   | NOT NULL                    |
| gender     | ENUM('Male','Female','Other') | NOT NULL                    |
| age        | INT                           | CHECK (age > 0)             |
| phone      | VARCHAR(15)                   | UNIQUE, NOT NULL            |
| email      | VARCHAR(100)                  | UNIQUE                      |
| address    | VARCHAR(150)                  | —                           |

### 6.2 buses

| Column         | Type                                         | Constraint                  |
|----------------|----------------------------------------------|-----------------------------|
| b_id           | INT                                          | PRIMARY KEY, AUTO_INCREMENT |
| bus_number     | VARCHAR(20)                                  | UNIQUE, NOT NULL            |
| bus_name       | VARCHAR(50)                                  | NOT NULL                    |
| bus_type       | ENUM('AC','Non-AC','Sleeper','Semi-Sleeper') | NOT NULL                    |
| total_seats    | INT                                          | CHECK (total_seats > 0)     |
| source         | VARCHAR(50)                                  | NOT NULL                    |
| destination    | VARCHAR(50)                                  | NOT NULL                    |
| departure_time | TIME                                         | —                           |
| arrival_time   | TIME                                         | —                           |

### 6.3 conductors

| Column     | Type        | Constraint                  |
|------------|-------------|-----------------------------|
| c_id       | INT         | PRIMARY KEY, AUTO_INCREMENT |
| first_name | VARCHAR(50) | NOT NULL                    |
| last_name  | VARCHAR(50) | NOT NULL                    |
| phone      | VARCHAR(15) | UNIQUE, NOT NULL            |

| Column           | Type         | Constraint                    |
|------------------|--------------|-------------------------------|
| email            | VARCHAR(100) | UNIQUE                        |
| experience_years | INT          | CHECK (experience_years >= 0) |

## 6.4 tickets

| Column        | Type         | Constraint                  |
|---------------|--------------|-----------------------------|
| t_id          | INT          | PRIMARY KEY, AUTO_INCREMENT |
| ticket_number | VARCHAR(20)  | UNIQUE, NOT NULL            |
| booking_date  | DATE         | NOT NULL                    |
| journey_date  | DATE         | NOT NULL                    |
| seat_number   | VARCHAR(10)  | NOT NULL                    |
| fare          | DECIMAL(8,2) | CHECK (fare > 0)            |

## 6.5 Junction (Relationship) Tables

The many-to-many and cross-entity relationships identified in the ER diagram are implemented using junction tables that hold foreign keys to the related entities:

- passenger\_ticket — links passengers to the tickets they have purchased (Buy)
- passenger\_bus — links passengers to the buses they travel on (Travel)
- conductor\_bus — links conductors to the buses they are assigned to (Have)
- conductor\_ticket — links conductors to the tickets they check (Check)

## Chapter 7: Normalization

---

### 7.1 First Normal Form (1NF)

Every table in the schema satisfies 1NF: all attributes hold atomic, single-valued data (e.g., a passenger's phone number is stored as one value per row), and there are no repeating groups or multi-valued columns.

### 7.2 Second Normal Form (2NF)

All non-key attributes are fully functionally dependent on the entire primary key. Since every table uses a single-column surrogate key (`p_id`, `b_id`, `c_id`, `t_id`), there are no partial dependencies — satisfying 2NF automatically.

### 7.3 Third Normal Form (3NF)

There are no transitive dependencies among non-key attributes. For example, in the buses table, `departure_time` and `arrival_time` depend only on `bus_id`, not on any other non-key attribute such as `bus_name` or `bus_type`. Relationship data (e.g., which passenger booked which ticket) is factored out into separate junction tables rather than being embedded redundantly within the entity tables.

Result: The JourneyJet database is normalized up to the Third Normal Form (3NF), eliminating redundancy, preventing update/insertion/deletion anomalies, and ensuring each fact is stored exactly once.

## Chapter 8: SQL Implementation

### 8.1 CREATE TABLE Statements

```
CREATE TABLE passengers (  
    p_id INT PRIMARY KEY AUTO_INCREMENT,  
    first_name VARCHAR(50) NOT NULL,  
    last_name VARCHAR(50) NOT NULL,  
    gender ENUM('Male','Female','Other') NOT NULL,  
    age INT CHECK (age > 0),  
    phone VARCHAR(15) UNIQUE NOT NULL,  
    email VARCHAR(100) UNIQUE,  
    address VARCHAR(150)  
);  
  
CREATE TABLE buses (  
    b_id INT PRIMARY KEY AUTO_INCREMENT,  
    bus_number VARCHAR(20) UNIQUE NOT NULL,  
    bus_name VARCHAR(50) NOT NULL,  
    bus_type ENUM('AC','Non-AC','Sleeper','Semi-Sleeper') NOT NULL,  
    total_seats INT CHECK (total_seats > 0),  
    source VARCHAR(50) NOT NULL,  
    destination VARCHAR(50) NOT NULL,  
    departure_time TIME,  
    arrival_time TIME  
);  
  
CREATE TABLE conductors (  
    c_id INT PRIMARY KEY AUTO_INCREMENT,  
    first_name VARCHAR(50) NOT NULL,  
    last_name VARCHAR(50) NOT NULL,  
    phone VARCHAR(15) UNIQUE NOT NULL,  
    email VARCHAR(100) UNIQUE,  
    experience_years INT CHECK (experience_years >= 0)  
);  
  
CREATE TABLE tickets (  
    t_id INT PRIMARY KEY AUTO_INCREMENT,  
    ticket_number VARCHAR(20) UNIQUE NOT NULL,  
    booking_date DATE NOT NULL,  
    journey_date DATE NOT NULL,  
    seat_number VARCHAR(10) NOT NULL,  
    fare DECIMAL(8,2) CHECK (fare > 0)  
);  
  
CREATE TABLE passenger_bus (  
    p_id INT, b_id INT,  
    PRIMARY KEY (p_id, b_id),  
    FOREIGN KEY (p_id) REFERENCES passengers(p_id) ON DELETE CASCADE,  
    FOREIGN KEY (b_id) REFERENCES buses(b_id) ON DELETE CASCADE  
);  
  
CREATE TABLE passenger_ticket (  
    p_id INT, t_id INT,  
    PRIMARY KEY (p_id, t_id),  
    FOREIGN KEY (p_id) REFERENCES passengers(p_id) ON DELETE CASCADE,
```

```

    FOREIGN KEY (t_id) REFERENCES tickets(t_id) ON DELETE CASCADE
);

CREATE TABLE conductor_bus (
    c_id INT, b_id INT,
    PRIMARY KEY (c_id, b_id),
    FOREIGN KEY (c_id) REFERENCES conductors(c_id) ON DELETE CASCADE,
    FOREIGN KEY (b_id) REFERENCES buses(b_id) ON DELETE CASCADE
);

CREATE TABLE conductor_ticket (
    c_id INT, t_id INT,
    PRIMARY KEY (c_id, t_id),
    FOREIGN KEY (c_id) REFERENCES conductors(c_id) ON DELETE CASCADE,
    FOREIGN KEY (t_id) REFERENCES tickets(t_id) ON DELETE CASCADE
);

```

## 8.2 Sample Queries

### 8.2.1 Basic Selection

```

SELECT * FROM passengers;
SELECT * FROM buses;
SELECT * FROM tickets;

```

### 8.2.2 Join Query — Passenger Bus Details

```

SELECT p.first_name, p.last_name, b.bus_name, b.source, b.destination
FROM passenger_bus pb
JOIN passengers p ON pb.p_id = p.p_id
JOIN buses b ON pb.b_id = b.b_id;

```

### 8.2.3 Join Query — Passenger Ticket Details

```

SELECT p.first_name, p.last_name, t.ticket_number, t.journey_date, t.fare
FROM passenger_ticket pt
JOIN passengers p ON pt.p_id = p.p_id
JOIN tickets t ON pt.t_id = t.t_id;

```

### 8.2.4 Nested Query — Passengers Booking Above-Average Fare

```

SELECT p.first_name, p.last_name, t.fare
FROM passengers p
JOIN passenger_ticket pt ON p.p_id = pt.p_id
JOIN tickets t ON pt.t_id = t.t_id
WHERE t.fare > (SELECT AVG(fare) FROM tickets);

```

### 8.2.5 Aggregate Query — Revenue per Bus Route

```

SELECT b.source, b.destination, COUNT(t.t_id) AS tickets_sold, SUM(t.fare) AS
total_revenue
FROM buses b
JOIN passenger_bus pb ON b.b_id = pb.b_id
JOIN passenger_ticket pt ON pb.p_id = pt.p_id
JOIN tickets t ON pt.t_id = t.t_id
GROUP BY b.b_id
HAVING SUM(t.fare) > 5000;

```

### 8.2.6 View — Conductor Bus Summary

```
CREATE VIEW conductor_bus_summary AS
SELECT c.first_name, c.last_name, b.bus_name, b.source, b.destination
FROM conductor_bus cb
JOIN conductors c ON cb.c_id = c.c_id
JOIN buses b ON cb.b_id = b.b_id;
```

## Chapter 9: Testing

The following test cases were executed across all modules to validate the correctness of CRUD operations and the reporting module.

| Test Case                     | Module    | Expected Result                           | Status |
|-------------------------------|-----------|-------------------------------------------|--------|
| Add Passenger                 | Passenger | Record inserted into passengers table     | Pass   |
| Update Passenger              | Passenger | Record updated successfully               | Pass   |
| Delete Passenger              | Passenger | Record deleted, related bookings cascaded | Pass   |
| Search Passenger              | Passenger | Correct record(s) displayed               | Pass   |
| Add Bus                       | Bus       | Record inserted into buses table          | Pass   |
| Update Bus                    | Bus       | Bus details updated successfully          | Pass   |
| Add Conductor                 | Conductor | Record inserted into conductors table     | Pass   |
| Add Ticket                    | Ticket    | Ticket inserted with fare and seat number | Pass   |
| Generate Passenger Bus Report | Reports   | Joined report displayed correctly         | Pass   |
| Generate Conductor Bus Report | Reports   | Joined report displayed correctly         | Pass   |
| Duplicate Phone Number        | Passenger | Insertion rejected (UNIQUE constraint)    | Pass   |

## Chapter 10: Results

The developed Bus Reservation System (JourneyJet) successfully manages the following volume of data, as reflected on the live dashboard:

| Entity     | Total Records |
|------------|---------------|
| Passengers | 103           |
| Buses      | 100           |
| Conductors | 100           |
| Tickets    | 100           |

Through this implementation, the project successfully demonstrates the following core DBMS concepts in a practical, working system:

- ER Modeling — translating real-world entities into a formal ER diagram
- Relational Schema Design — converting the ER model into normalized tables
- Normalization — achieving 3NF across all entity tables
- CRUD Operations — Add, Update, Delete, and Search across every module
- Joins — combining Passenger, Bus, Conductor, and Ticket data for reporting
- Constraints — primary keys, foreign keys, UNIQUE, and CHECK constraints

- Reporting — consolidated, multi-table views generated through SQL

## Future Scope

---

- Integration of an online payment gateway for ticket purchases
- Interactive seat selection and visualization on the booking screen
- User authentication and role-based access (Admin vs. Staff vs. Passenger)
- A companion mobile application for passengers
- QR-code based digital tickets for faster boarding verification
- Real-time GPS-based bus tracking integrated with the existing schema

## Conclusion

---

The Bus Reservation System, JourneyJet, successfully automates the management of transportation records using a normalized MySQL database and a structured web-based interface. By replacing manual, paper-based processes with a centralized relational database, the system significantly reduces the chances of data inconsistency and human error.

Through the design and implementation of this project, key Database Management System concepts — including ER modeling, relational schema design, normalization up to 3NF, primary/foreign key constraints, and advanced SQL operations such as joins, nested queries, and aggregate functions — were applied to a real, functioning system. The result is a database-driven application that ensures data integrity, reduces manual effort, and provides efficient, on-demand reporting across all entities in the system.

## References

---

- [1] Elmasri, R., & Navathe, S. B. — Fundamentals of Database Systems. Pearson Education.
- [2] MySQL 8.0 Reference Manual — <https://dev.mysql.com/doc/>
- [3] Node.js / Express.js Official Documentation — <https://expressjs.com/>
- [4] W3Schools — HTML, CSS, JavaScript and SQL Tutorials — <https://www.w3schools.com/>
- [5] Stack Overflow — reference for resolving frontend/backend integration issues — <https://stackoverflow.com/>